In[751]:=

```
ClearAll["Global`*"];



P59[a_, b_, n_Integer, s_Integer] := If[
n ≤ 0,
{DirectedEdge[a, b]},
DirectedEdge @@@ Partition[Join[{a}, Range[s, s + n - 1], {b}], 2, 1]
];



A59[t_, fan_Integer, s_Integer] := Module[{p1, p2, ls},
p1 = s + 1;
p2 = s + 2;
ls = Range[s + 3, s + 2 + fan];
Join[
{DirectedEdge[p2, p1], DirectedEdge[p1, t]},
Table[DirectedEdge[ls[[i]], p2], {i, Length[ls]}]
]
];



T59[d_Integer, r_String, a_Integer, seed_Integer] := Module[
{bb, K, c, v, q, e, f = {}, ib, m, safe, u, dum, r1, r2, wrong,
main, perm, anc, i, j},

bb = 1 000 000 000 * seed;
K = bb + 1;

c = Table[bb + 100 + i, {i, 4}];
v = Table[bb + 200 + i, {i, 4}];
q = Table[bb + 300 + i, {i, 4}];

e = Flatten[
Table[
{DirectedEdge[K, c[[i]]], DirectedEdge[c[[i]], v[[i]]]},
{i, 4}
], 1
];
```

```
Do[
ib = bb + 20 000 000 * i;

m = ib + 1;
safe = ib + 2;
u = ib + 3;
dum = ib + 4;
r1 = ib + 10;
r2 = ib + 20;

wrong = c[[1 + Mod[i, 4]]];

main = Join[
P59[q[[i]], r1, d, ib + 1 000 000],
P59[q[[i]], r2, d, ib + 2 000 000],
{DirectedEdge[r1, m], DirectedEdge[r2, m]},
P59[q[[i]], safe, d + 1, ib + 3 000 000]
];

perm = If[
r === "Continue",
{
DirectedEdge[m, c[[i]]],
DirectedEdge[safe, dum],
DirectedEdge[u, wrong]
},
{
DirectedEdge[m, wrong],
DirectedEdge[safe, c[[i]]],
DirectedEdge[u, dum]
}
];

anc = Join[
A59[m, i, bb + 970 000 000 + 10 000 * i],
A59[c[[i]], i, bb + 980 000 000 + 10 000 * i]
];

e = Join[e, main, perm, anc];
AppendTo[f, m],
{i, 4}
];
```

```
{{Union[e], q, K, v, c, f}, a}
];


Case59[d_Integer, a_Integer, r_String] :=
T59[d, r, a, 59 000 000 + 100 * d + a];
```

In[756]:=

```
ClearAll[Deg59];



Deg59[c_List] := Module[{e, v, ic, oc},
e = c[[1, 1]];
v = Union[Flatten[List @@@ e]];
ic = Counts[Cases[e, DirectedEdge[x_, y_] :> y]];
oc = Counts[Cases[e, DirectedEdge[x_, y_] :> x]];
AssociationThread[
v,
Table[{Lookup[ic, z, 0], Lookup[oc, z, 0]}, {z, v}]
]
];



audit59 = Flatten[
Table[
Module[{c, s},
c = Case59[d, a, "Continue"];
s = Case59[d, a, "Stop"];
<|
"Depth" → d,
"Answer" → a,
"DegreeMapSame" → SameQ[Deg59[c], Deg59[s]],
"ContinueOnlyEdges" → Length[
Complement[c[[1, 1]], s[[1, 1]]]
],
"StopOnlyEdges" → Length[
Complement[s[[1, 1]], c[[1, 1]]]
]
|>
],
{d, {2, 5, 9, 15}},
{a, 1, 4}
],
1
];



Dataset[audit59]
```

Out[759]=

Depth	Answer	DegreeMapSame	ContinueOnlyEdges	StopOnlyEdges
2	1	True	12	12
2	2	True	12	12
2	3	True	12	12
2	4	True	12	12
5	1	True	12	12
5	2	True	12	12
5	3	True	12	12
5	4	True	12	12
9	1	True	12	12
9	2	True	12	12
9	3	True	12	12
9	4	True	12	12
15	1	True	12	12
15	2	True	12	12
15	3	True	12	12
15	4	True	12	12

In[760]:=

```
ClearAll[Reach59];



Reach59[c_List, mode_String] := Module[
{e, q, ca, ans, f, front, seen = {}, next, u, out, reached},

e = c[[1, 1]];
q = c[[1, 2, c[[2]]]];
ca = c[[1, 5]];
ans = c[[2]];
f = c[[1, 6]];

front = {q};

While[Length[front] > 0,
u = First[front];
front = Rest[front];

If[! MemberQ[seen, u],
```

```
AppendTo[seen, u];

out = If[
mode === "Stop" && MemberQ[f, u],
{},
Cases[e, DirectedEdge[u, v_] :> v]
];

next = Complement[out, seen];
front = Join[front, next]
]
];

reached = Select[
Range[4],
MemberQ[seen, ca[[#]]] &
];

Boole[
Length[reached] === 1 && First[reached] === ans
]
];



necessity59 = Flatten[
Table[
Module[{c, s},
c = Case59[d, a, "Continue"];
s = Case59[d, a, "Stop"];

<|
"Depth" -> d,
"Answer" -> a,
"ContinuePropagate" -> Reach59[c, "Propagate"],
"ContinueStop" -> Reach59[c, "Stop"],
"StopPropagate" -> Reach59[s, "Propagate"],
"StopStop" -> Reach59[s, "Stop"]
|>
],
{d, {2, 5, 9, 15}},
{a, 1, 4}
],
1
```

```
];
```

```
Dataset[necessity59]
```

Out[763]=

Depth	Answer	ContinuePropagate	ContinueStop	StopPropagate	StopStop
2	1	1	0	0	1
2	2	1	0	0	1
2	3	1	0	0	1
2	4	1	0	0	1
5	1	1	0	0	1
5	2	1	0	0	1
5	3	1	0	0	1
5	4	1	0	0	1
9	1	1	0	0	1
9	2	1	0	0	1
9	3	1	0	0	1
9	4	1	0	0	1
15	1	1	0	0	1
15	2	1	0	0	1
15	3	1	0	0	1
15	4	1	0	0	1

In[764]:=

```
cert59 = <|
"Stage" → "S59A",
"Pairs" → Length[audit59],
"ExactDegreeBalanced" → Count[
audit59, x_ /; TrueQ[x["DegreeMapSame"]]
],
"OppositeActionNecessary" → Count[
necessity59,
x_ /;
x["ContinuePropagate"] == 1 &&
x["ContinueStop"] == 0 &&
x["StopPropagate"] == 0 &&
x["StopStop"] == 1
],
"CoreTCCTChanged" → False,
"UnionSemantics" → "SingleFinalGlobalUnion"
|>;
```

```
Dataset[{cert59}]
```

Out[765]=

Stage	Pairs	ExactDegreeBalanced	OppositeActionNecessary	CoreTCCTChanged	UnionSemantics
S59A	16	16	16	False	SingleFinalGlobalU

In[766]:=

```
ClearAll[Feat59, Row59, Acc59, Fit59];
```

```
Feat59[c_List] := Module[{e, ca, dm, v, cd, ad},
e = c[[1, 1]];
ca = c[[1, 5]];
dm = Deg59[c];
v = Keys[dm];
cd = Lookup[dm, ca];
ad = Values[dm];

<|
"CandInTotal" → Total[cd[[All, 1]]],
"CandOutTotal" → Total[cd[[All, 2]]],
"CandMerge" → Count[cd[[All, 1]], x_ /; x ≥ 2],
"CandSplit" → Count[cd[[All, 2]], x_ /; x ≥ 2],
"CandInMax" → Max[cd[[All, 1]]],
```

```
"CandOutMax" → Max[cd[[All, 2]]],
"CandPairTypes" → Length@DeleteDuplicates[cd],
"Vertices" → Length[v],
"Edges" → Length[e],
"MergeNodes" → Count[ad[[All, 1]], x_/; x ≥ 2],
"SplitNodes" → Count[ad[[All, 2]], x_/; x ≥ 2],
"Sources" → Count[ad[[All, 1]], 0],
"Sinks" → Count[ad[[All, 2]], 0],
"DegreePairTypes" → Length@DeleteDuplicates[ad]
|>
];



Row59[d_, a_, r_] := Join[
<|"Depth" → d, "Answer" → a, "Target" → r|> ,
Feat59[Case59[d, a, r]]
];



train59 = Flatten[
Table[
Row59[d, a, r],
{d, {2, 5}},
{a, 1, 4},
{r, {"Continue", "Stop"}}
], 2];



held59 = Flatten[
Table[
Row59[d, a, r],
{d, {9, 15}},
{a, 1, 4},
{r, {"Continue", "Stop"}}
], 2];



featNames59 = Keys@KeyDrop[
First[train59],
{"Depth", "Answer", "Target"}
```

```
];



Acc59[rows_, f_, t_, dir_] := N@Mean[
Function[x,
Boole[
SameQ[
If[dir == 1, x[f] > t, x[f] ≤ t],
x["Target"] === "Continue"
]
]
]/@ rows
];



Fit59[f_, idx_] := Module[{v, ths, cand, best},
v = Sort@DeleteDuplicates[Lookup[train59, f]];

ths = If[
Length[v] == 1,
{First[v] - 1, First[v] + 1},
Join[
{First[v] - 1},
Table[N[(v[[i]] + v[[i + 1]]) / 2], {i, Length[v] - 1}],
{Last[v] + 1}
]
];

cand = Flatten[Table[{t, d}, {t, ths}, {d, {1, -1}}], 1];

best = First@SortBy[
cand,
Function[q, {- Acc59[train59, f, q[[1]], q[[2]]], q[[1]], q[[2]]}]
];

<|
"Feature" → f,
"Index" → idx,
"Threshold" → best[[1]],
"Direction" → best[[2]],
"TrainAccuracy" → Acc59[train59, f, best[[1]], best[[2]],
```

```
"HeldoutAccuracy" → Acc59[held59, f, best[[1]], best[[2]]]
|>
];
```

```
fits59 = Table[
Fit59[featNames59[[i]], i],
{i, Length[featNames59]}
];
```

```
rank59 = SortBy[
fits59,
Function[x, {-x["TrainAccuracy"], x["Index"]}]
];
```

```
Dataset[rank59]
```

Out[776]=

Feature	Index	Threshold	Direction	TrainAccuracy	HeldoutAccuracy
CandInTotal	1	15	−1	0.5	0.5
CandOutTotal	2	3	−1	0.5	0.5
CandMerge	3	3	−1	0.5	0.5
CandSplit	4	−1	−1	0.5	0.5
CandInMax	5	3	−1	0.5	0.5
CandOutMax	6	0	−1	0.5	0.5
CandPairTypes	7	0	−1	0.5	0.5
Vertices	8	100	−1	0.5	0.5
Edges	9	103	−1	0.5	0.5
MergeNodes	10	13	−1	0.5	0.5
SplitNodes	11	4	−1	0.5	0.5
Sources	12	28	−1	0.5	0.5
Sinks	13	7	−1	0.5	0.5
DegreePairTypes	14	7	−1	0.5	0.5

In[779]:=

```
best59 = First[rank59];
```

```
cert59B = <|
"Stage" → "S59B",
"Method" → "ZeroTCCTShortcutGate",
"TrainCases" → Length[train59],
"HeldoutCases" → Length[held59],
"FeaturesTested" → Length[featNames59],
"BestFeature" → best59["Feature"],
"TrainAccuracy" → best59["TrainAccuracy"],
"HeldoutAccuracy" → best59["HeldoutAccuracy"],
"TCCTControllerUsed" → False,
"Retraining" → False
|>;
```

```
Dataset[{cert59B}]
```

Out[781]=

Stage	Method	TrainCases	HeldoutCases	FeaturesTested
S59B	ZeroTCCTShortcutGate	16	16	14

In[782]:=

```
ClearAll[Struct59, JointCode59, WLPair59];
```

```
Struct59[c_List] := Module[{e, v, gi, go},
e = c[[1, 1]];
v = Union@Join[
Flatten[List @@@ e],
c[[1, 2]], {c[[1, 3]]}, c[[1, 4]], c[[1, 5]], c[[1, 6]]
];
gi = GroupBy[Cases[e, DirectedEdge[x_, y_] :→ {y, x}], First → Last];
go = GroupBy[Cases[e, DirectedEdge[x_, y_] :→ {x, y}], First → Last];
{
v,
AssociationThread[v, Lookup[gi, v, {}]],
AssociationThread[v, Lookup[go, v, {}]]
}
];
```

```
JointCode59[a_List, b_List] := Module[{u, m},
u = DeleteDuplicates[Join[a, b]];
m = AssociationThread[HoldComplete /@ u, Range[Length[u]]];
{
Lookup[m, HoldComplete /@ a],
Lookup[m, HoldComplete /@ b]
}
];



WLPair59[c_List, s_List, rmax_Integer] := Module[
{gc, gs, vc, vs, pc, ps, cc, cs, tc, ts, jc, lc, ls, hc, hs, k, x},

gc = Struct59[c]; gs = Struct59[s];

vc = gc[[1]]; pc = gc[[2]]; cc = gc[[3]];
vs = gs[[1]]; ps = gs[[2]]; cs = gs[[3]];

tc = Table[
{Length[Lookup[pc, x, {}]], Length[Lookup[cc, x, {}]]},
{x, vc}
];

ts = Table[
{Length[Lookup[ps, x, {}]], Length[Lookup[cs, x, {}]]},
{x, vs}
];

jc = JointCode59[tc, ts];
lc = AssociationThread[vc, jc[[1]]];
ls = AssociationThread[vs, jc[[2]]];

hc = {lc}; hs = {ls};

Do[
tc = Table[
{
Lookup[lc, x],
Sort@Lookup[lc, Lookup[pc, x, {}]],
Sort@Lookup[lc, Lookup[cc, x, {}]]
},
{x, vc}
```

```
];

ts = Table[
{
Lookup[ls, x],
Sort@Lookup[ls, Lookup[ps, x, {}]],
Sort@Lookup[ls, Lookup[cs, x, {}]]
},
{x, vs}
];

jc = JointCode59[tc, ts];
lc = AssociationThread[vc, jc[[1]]];
ls = AssociationThread[vs, jc[[2]]];

AppendTo[hc, lc];
AppendTo[hs, ls],
{k, rmax}
];

{hc, hs}
];
```

In[786]:=

```
ClearAll[Ident59];



Ident59[d_Integer, a_Integer] := Module[
{c, s, h, cc, cs, cnt, gd, fr, fg, r, i},

c = Case59[d, a, "Continue"];
s = Case59[d, a, "Stop"];

h = WLPair59[c, s, 8];

cc = c[[1, 5]];
cs = s[[1, 5]];

cnt = Table[
Total@Table[
Boole[
Lookup[h[[1, r + 1]], cc[[i]], -1] =!=
Lookup[h[[2, r + 1]], cs[[i]], -2]
],
```

```
{i, 4}
],
{r, 0, 8}
];

gd = Table[
Counts[Values[h[[1, r + 1]]]] =!=
Counts[Values[h[[2, r + 1]]]],
{r, 0, 8}
];

fr = FirstCase[
Range[0, 8],
x_ /; cnt[[x + 1]] > 0,
Missing["None"]
];

fg = FirstCase[
Range[0, 8],
x_ /; TrueQ[gd[[x + 1]]],
Missing["None"]
];

<|
"Depth" → d,
"Answer" → a,
"CandidateDiffR0" → cnt[[1]],
"FirstCandidateRadius" → fr,
"FirstGlobalRadius" → fg
|>
];



ident59 = Flatten[
Table[
Ident59[d, a],
{d, {2, 5, 9, 15}},
{a, 1, 4}
],
1
];
```

```
Dataset[ident59]
```

Out[789]=

Depth	Answer	CandidateDiffR0	FirstCandidateRadius	FirstGlobalRadius
2	1	0	1	1
2	2	0	1	1
2	3	0	1	1
2	4	0	1	1
5	1	0	1	1
5	2	0	1	1
5	3	0	1	1
5	4	0	1	1
9	1	0	1	1
9	2	0	1	1
9	3	0	1	1
9	4	0	1	1
15	1	0	1	1
15	2	0	1	1
15	3	0	1	1
15	4	0	1	1

In[790]:=

```
cert59C = <|
"Stage" → "S59C",
"Pairs" → Length[ident59],
"Radius0CandidateLeak" → Count[
ident59, x_; x["CandidateDiffR0"] > 0
],
"IdentifiableByR8" → Count[
ident59, x_; ! MissingQ[x["FirstCandidateRadius"]]
],
"GlobalLeakAtR0" → Count[
ident59, x_; TrueQ[x["FirstGlobalRadius"] === 0]
],
"CandidateFirstRadiusCounts" → Counts[
Lookup[ident59, "FirstCandidateRadius"]
],
"GlobalFirstRadiusCounts" → Counts[
Lookup[ident59, "FirstGlobalRadius"]
],
"CoreTCCTChanged" → False,
"UnionSemantics" → "SingleFinalGlobalUnion"
|>;
```

```
Dataset[{cert59C}]
```

Out[791]=

Stage	Pairs	Radius0CandidateLeak	IdentifiableByR8	GlobalLeakAtR0	CandidateFirstRadiusCounts	
S59C	16	0	16	0	1	16

In[792]:=

```
ClearAll[Pack60, Inv60, Prep60];
```

```
rw60 = {-9, 6, -2, -2, 4, 2, -2, 6, 0};
enc60 = {3, 6};
prop60 = {{0, 0, 1, 0, 0, 1}};

Pack60[c_List] := Module[
{x = c[[1]], a = c[[2]], e, v, pos, gi, go, pa, ch, id, od, pin, pout, cin, cout},
e = x[[1]];
v = Union@Join[Flatten[List @@@ e], x[[2]], {x[[3]]}, x[[4]], x[[5]], x[[6]]];
pos = AssociationThread[v, Range[Length[v]]];
```

```
gi = GroupBy[Cases[e, DirectedEdge[u_, w_] :> {w, u}], First -> Last];
go = GroupBy[Cases[e, DirectedEdge[u_, w_] :> {u, w}], First -> Last];
pa = (Lookup[pos, Lookup[gi, #, {}]] &) /@ v;
ch = (Lookup[pos, Lookup[go, #, {}]] &) /@ v;
id = Length /@ pa;
od = Length /@ ch;
pin = (Total[id[[#]] &) /@ pa;
pout = (Total[od[[#]] &) /@ pa;
cin = (Total[id[[#]] &) /@ ch;
cout = (Total[od[[#]] &) /@ ch;
{
pa, id, od,
Lookup[pos, x[[2, a]]],
Lookup[pos, x[[5]]],
a, Length[v],
pin, pout, cin, cout, v
}
];

Inv60[c_List] := Module[
{p, pa, id, od, n, ch, h, sig, sigs, j, k, pp},
p = Pack60[c];
pa = p[[1]];
id = p[[2]];
od = p[[3]];
n = p[[7]];
ch = Table[{}, n];

Do[
Do[
If[pp > 0, ch[[pp]] = Append[ch[[pp]], j]],
{pp, pa[[j]]}
],
{j, n}
];

h = MapThread[2 Boole[#1 ≥ 2] + Boole[#2 ≥ 2] &, {id, od}];

sig[state_, u_] := Module[{ps, cs, din, dout, min, mout},
ps = state[[pa[[u]]]];
cs = state[[ch[[u]]]];
din = Length@DeleteDuplicates[ps];
dout = Length@DeleteDuplicates[cs];
```

```
min = If[Length[ps] == 0, 0, Max @ Values @ Counts[ps]];
mout = If[Length[cs] == 0, 0, Max @ Values @ Counts[cs]];
{
Boole[din > 1],
Boole[dout > 1],
Boole[min > 1],
Boole[mout > 1],
Boole[MemberQ[ps, state[[u]]]],
Boole[MemberQ[cs, state[[u]]]
}
];

Do[
sigs = Table[sig[h, j], {j, n}];
h = (2 #[[enc60[[1]]]] + #[[enc60[[2]]]]) & /@ sigs,
{k, 3}
];

Table[sig[h, j], {j, n}]
];

Prep60[c_List] := Module[{p, s},
p = Pack60[c];
s = Inv60[c];
{p, AssociationThread[p[[12]], s]}
];
```

In[799]:=

```
ClearAll[Run60];



Run60[x_List, prop_List, ret_Integer : 0] := Module[
{p = x[[1]], wl = x[[2]], pa, id, od, q, ca, ans, n, pin, pout, cin, cout, v,
h, z, h2, nz, arr, j, k, s, pp, ap, pz, o, zv, rf, rd, dd, pg, allow,
active, reached, pass, code, blocks = {}},

pa = p[[1]]; id = p[[2]]; od = p[[3]];
q = p[[4]]; ca = p[[5]]; ans = p[[6]]; n = p[[7]];
pin = p[[8]]; pout = p[[9]]; cin = p[[10]]; cout = p[[11]];
v = p[[12]];

h = ConstantArray[0, n];
z = ConstantArray[0, n];
arr = ConstantArray[-1, n];
```

```
h[[q]] = 1;
arr[[q]] = 0;

For[k = 1, k ≤ 64, k++,
h2 = ConstantArray[0, n];
nz = ConstantArray[0, n];
active = 0;

Do[
ap = 0; pz = 0;

Do[
pp = pa[[j, s]];
If[pp > 0 && h[[pp]] == 1,
ap++;
If[z[[pp]] > pz, pz = z[[pp]]]
],
{s, Length[pa[[j]]]}
];

If[ap > 0,
o = Boole[ap > 1];
zv = {0, 1, 2, 1, 3, 1, 3, 1}[[1 + 2 * pz + o]];
nz[[j]] = zv;

If[arr[[j]] < 0, arr[[j]] = k];

rf = {
1, Boole[zv > 0], zv,
Sign[pin[[j]] - id[[j]]],
Sign[pout[[j]] - id[[j]]],
Sign[cin[[j]] - id[[j]]],
Sign[cout[[j]] - od[[j]]],
Sign[id[[j]] - od[[j]]],
Sign[pout[[j]] - cin[[j]]]
};

rd = rw60.rf;
dd = 1 - cin[[j]] + 2 * cout[[j]];
pg = od[[j]] + 2 * Boole[zv > 0] - zv;
code = Lookup[wl, v[[j]]];

If[ret == 1 && rd > 0 && dd ≤ 0, AppendTo[blocks, code]];
```

```
allow = If[
rd > 0 && dd ≤ 0 && MemberQ[prop, code],
pg > 0,
If[rd > 0, dd > 0, pg > 0]
];

If[TrueQ[allow],
h2[[j]] = 1;
active++
]
],
{j, n}
];

h = h2;
z = nz;

If[active == 0, Break[]]
];

reached = Select[
Range[4],
arr[[ca[[#]]]] ≥ 0 &
];

pass = Boole[
Length[reached] === 1 && First[reached] === ans
];

If[ret == 1, {pass, DeleteDuplicates[blocks]}, pass]
];
```

In[801]:=
```
x60 = Prep60[Case59[2, 1, "Continue"]];
```

```
{
Length[x60],
Length[x60[[1]]],
AssociationQ[x60[[2]]],
enc60,
prop60,
Length[First[prop60]],
MemberQ[{0, 1}, Run60[x60, {}]],
MemberQ[{0, 1}, Run60[x60, prop60]]
}
```

Out[802]=
```
{2, 12, True, {3, 6}, {{0, 0, 1, 0, 0, 1}}, 6, True, True}
```

In[803]:=
```
spec60 = Tuples[
{{2, 5, 9, 15}, Range[4], {"Continue", "Stop"}}
];
```

```
cases60 = (Prep60[Case59 @@ #] &) /@ spec60;
```

```
rows60 = MapThread[
Function[{sp, x},
<|
"Depth" → sp[[1]],
"Answer" → sp[[2]],
"Target" → sp[[3]],
"BaselinePass" → Run60[x, {}],
"FrozenPass" → Run60[x, prop60]
|>
],
{spec60, cases60}
];
```

```
sum60 = Flatten[
```

```
Table[
Module[{z},
z = Select[
rows60,
#["Depth"] === d && #["Target"] === t &
];
<|
"Depth" → d,
"Target" → t,
"Cases" → Length[z],
"BaselinePassed" → Total[Lookup[z, "BaselinePass"]],
"FrozenPassed" → Total[Lookup[z, "FrozenPass"]]
|>
],
{d, {2, 5, 9, 15}},
{t, {"Continue", "Stop"}}
],
1
];
```

```
Dataset[sum60]
```

Out[807]=

Depth	Target	Cases	BaselinePassed	FrozenPassed
2	Continue	4	0	0
2	Stop	4	4	4
5	Continue	4	0	0
5	Stop	4	4	4
9	Continue	4	0	0
9	Stop	4	4	4
15	Continue	4	0	0
15	Stop	4	4	4

In[808]:=

```
base60 = Total[Lookup[rows60, "BaselinePass"]];
frozen60 = Total[Lookup[rows60, "FrozenPass"]];

seen60 = Total[
Table[
r60 = Run60[cases60[[i]], {}, 1];
Boole[
MemberQ[
r60[[2]],
First[prop60]
]
],
{i, Length[cases60]}
]
];

cert60 = <|
"Stage" → "S60A",
"Benchmark" → "S59BalancedOppositeAction",
"Cases" → Length[cases60],
"ZeroTCCTReferenceAccuracy" → 0.5,
"LegacyBaselinePassed" → base60,
"LegacyBaselineAccuracy" → N[base60 / Length[cases60]],
"FrozenS52Passed" → frozen60,
"FrozenS52Accuracy" → N[frozen60 / Length[cases60]],
"SelectedPatternSeenCases" → seen60,
"Encoder" → enc60,
"ModelFrozen" → True,
"Retraining" → False,
"PolicySearch" → False,
"CoreTCCTChanged" → False,
"UnionSemantics" → "SingleFinalGlobalUnion"
|>;

Dataset[{cert60}]
```

Out[812]=

Stage	Benchmark	Cases	ZeroTCCTReferenceA
S60A	S59BalancedOppositeAction	32	0.5

columns 1–10 of **15**

In[813]:=

```
{
Length[rows60],
Length[cases60],
base60,
frozen60,
seen60
}
```

Out[813]=

```
{32, 32, 16, 16, 0}
```

In[814]:=

```
ClearAll[SigLevels61];



SigLevels61[c_List, rmax_Integer] := Module[
{p, pa, id, od, n, ch, cur, nxt, levs, j, k, u},

p = Pack60[c];
pa = p[[1]];
id = p[[2]];
od = p[[3]];
n = p[[7]];

ch = Table[{}, n];

Do[
Do[
If[u > 0, ch[[u]] = Append[ch[[u]], j]],
{u, pa[[j]]}
],
{j, n}
];

cur = AssociationThread[
Range[n],
MapThread[List, {id, od}]
];

levs = {cur};

Do[
nxt = AssociationThread[
Range[n],
```

```
Table[
{
Lookup[cur, j],
Sort[Lookup[cur, pa[[j]]]],
Sort[Lookup[cur, ch[[j]]]]
},
{j, n}
]
];

AppendTo[levs, nxt];
cur = nxt,
{k, rmax}
];

levs
];
```

In[816]:=

```
ClearAll[Reject61];



Reject61[c_List] := Module[
{p, pa, id, od, q, n, pin, pout, cin, cout, h, z, h2, nz,
j, k, aps, ap, pz, o, zv, rf, rd, dd, pg, allow, active, rej = {}},

p = Pack60[c];

pa = p[[1]];
id = p[[2]];
od = p[[3]];
q = p[[4]];
n = p[[7]];
pin = p[[8]];
pout = p[[9]];
cin = p[[10]];
cout = p[[11]];

h = ConstantArray[0, n];
z = ConstantArray[0, n];
h[[q]] = 1;

For[k = 1, k ≤ 64, k++,
```

```
h2 = ConstantArray[0, n];
nz = ConstantArray[0, n];
active = 0;

Do[
aps = Select[
pa[[j]],
Function[u, u > 0 && h[[u]] == 1]
];

ap = Length[aps];

If[ap > 0,

pz = Max[z[[aps]]];
o = Boole[ap > 1];

zv = {0, 1, 2, 1, 3, 1, 3, 1}[[1 + 2 * pz + o]];
nz[[j]] = zv;

rf = {
1,
Boole[zv > 0],
zv,
Sign[pin[[j]] - id[[j]]],
Sign[pout[[j]] - id[[j]]],
Sign[cin[[j]] - id[[j]]],
Sign[cout[[j]] - od[[j]]],
Sign[id[[j]] - od[[j]]],
Sign[pout[[j]] - cin[[j]]]
};

rd = rw60.rf;
dd = 1 - cin[[j]] + 2 * cout[[j]];
pg = od[[j]] + 2 * Boole[zv > 0] - zv;

If[
rd > 0 && dd ≤ 0,
AppendTo[rej, {k, j}]
];

allow = If[
rd > 0,
```

```
    dd > 0,
    pg > 0
    ];

    If[TrueQ[allow],
    h2[[j]] = 1;
    active++
    ]
    ],
    {j, n}
    ];

    h = h2;
    z = nz;

    If[active == 0, Break[]]
    ];

    rej
    ];
```

In[818]:=

```
ClearAll[Trace61];



Trace61[c_List, r_Integer] := Module[
{rej, lev, m},

rej = Reject61[c];

If[
Length[rej] == 0,
Return[{}]
];

lev = SigLevels61[c, r][[r + 1]];
m = Min[rej[[All, 1]]];

Sort[
Table[
{
rej[[i, 1]] - m,
Lookup[lev, rej[[i, 2]]]
},
{i, Length[rej]}
]
]
];
```

In[820]:=

```
c61 = Case59[2, 1, "Continue"];



{
Length[Reject61[c61]],
Length[SigLevels61[c61, 3]],
ListQ[Trace61[c61, 0]],
ListQ[Trace61[c61, 3]]
}
```

Out[821]=

```
{1, 4, True, True}
```

In[822]:=

```
audit61 = Cases[
Table[
Module[
{c, s},

c = Case59[d, a, "Continue"];
s = Case59[d, a, "Stop"];

Table[
<|
"Depth" → d,
"Answer" → a,
"Radius" → r,
"Same" → SameQ[
Trace61[c, r],
Trace61[s, r]
]
|>,
{r, 0, 3}
]
],
{d, {2, 5, 9, 15}},
{a, 1, 4}
],
_Association,
Infinity
];



{
Length[audit61],
AllTrue[
audit61,
Function[x, MemberQ[{True, False}, x["Same"]]]
]
}
```

Out[823]=

```
{64, True}
```

In[824]:=

```
summary61 = Table[
Module[{z},
z = Select[
audit61,
#["Radius"] === r &
];

<|
"Radius" → r,
"Pairs" → Length[z],
"Different" → Count[
z, x_/; FalseQ[x["Same"]]
],
"Same" → Count[
z, x_/; TrueQ[x["Same"]]
]
|>
],
{r, 0, 3}
];
```

```
Dataset[summary61]
```

Out[825]=

Radius	Pairs	Different	Same
0	16	0	16
1	16	0	16
2	16	0	0
3	16	0	0

In[826]:=

```
perfectR61 = FirstCase[
summary61,
x_ /; x["Different"] == 16 :> x["Radius"],
Missing["None"]
];



cert61 = <|
"Stage" -> "S61A",
"Benchmark" -> "S59BalancedOppositeAction",
"Pairs" -> 16,
"FirstPerfectDecisionRadius" -> perfectR61,
"Radius0Different" -> summary61[[1, "Different"]],
"Radius1Different" -> summary61[[2, "Different"]],
"Radius2Different" -> summary61[[3, "Different"]],
"Radius3Different" -> summary61[[4, "Different"]],
"Training" -> False,
"CoreTCCTChanged" -> False,
"UnionSemantics" -> "SingleFinalGlobalUnion"
|>;



Dataset[{cert61}]
```

Out[828]=

Stage	Benchmark	Pairs	FirstPerfectDecisionR
S61A	S59BalancedOppositeAction	16	—

columns 1–10 of **11**

In[829]:=

```
check61 = Table[
Module[{z},
z = Select[audit61, #["Radius"] === r &];
<|
"Radius" → r,
"Rows" → Length[z],
"True" → Count[z, x_/; TrueQ[x["Same"]]],
"False" → Count[z, x_/; FalseQ[x["Same"]]],
"Other" → Count[z, x_/; !MemberQ[{True, False}, x["Same"]]],
"OtherValues" → DeleteDuplicates[
Lookup[
Select[z, !MemberQ[{True, False}, #["Same"]] &],
"Same"
]
]
|>
],
{r, 0, 3}
];
```

```
Dataset[check61]
```

Out[830]=

Radius	Rows	True	False	Other	OtherValues
0	16	16	0	0	—
1	16	16	0	0	—
2	16	0	0	0	—
3	16	0	0	0	—

In[831]:=

```
ClearAll[Pair61B];



Pair61B[d_Integer, a_Integer, r_Integer] := Module[
{c, s, tc, ts, b},

c = Case59[d, a, "Continue"];
s = Case59[d, a, "Stop"];

tc = Trace61[c, r];
ts = Trace61[s, r];

b = Boole[SameQ[tc, ts]];

<|
"Depth" → d,
"Answer" → a,
"Radius" → r,
"SameCode" → b
|>
];



audit61B = Flatten[
Table[
Pair61B[d, a, r],
{d, {2, 5, 9, 15}},
{a, 1, 4},
{r, 0, 3}
],
2
];



{
Length[audit61B],
Count[audit61B, x_ /; MemberQ[{0, 1}, x["SameCode"]]]
}
```

Out[834]=

```
{64, 64}
```

In[835]:=

```
summary61B = Table[
Module[{z},
z = Select[audit61B, #["Radius"] === r &];
<|
"Radius" → r,
"Pairs" → Length[z],
"Different" → Count[z, x_/; x["SameCode"] == 0],
"Same" → Count[z, x_/; x["SameCode"] == 1]
|>
],
{r, 0, 3}
];
```

```
Dataset[summary61B]
```

Out[836]=

Radius	Pairs	Different	Same
0	16	0	16
1	16	0	16
2	16	16	0
3	16	16	0

In[837]:=

```
{
Length[audit61B],
And @@ Table[
summary61B[[i, "Different"]] + summary61B[[i, "Same"]] == 16,
{i, 4}
]
}
```

Out[837]=

```
{64, True}
```

In[838]:=

```
ClearAll[DecisionStates61C];
```

```
DecisionStates61C[c_List] := Module[
{rej, lev},
rej = Reject61[c];
lev = SigLevels61[c, 2][[3]];
```

```
DeleteDuplicates[
Lookup[lev, rej[[All, 2]]]
]
];



rows61C = Flatten[
Table[
{
<|
"Depth" → d,
"Answer" → a,
"Target" → "Continue",
"States" → DecisionStates61C[
Case59[d, a, "Continue"]
]
|>,
<|
"Depth" → d,
"Answer" → a,
"Target" → "Stop",
"States" → DecisionStates61C[
Case59[d, a, "Stop"]
]
|>
},
{d, {2, 5}},
{a, 1, 4}
],
2
];



Length[rows61C]
```

Out[841]=

```
16
```

In[842]:=

```
states61C = DeleteDuplicates@Flatten[
Lookup[rows61C, "States"],
1
];



conf61C = Table[
Module[{cc, ss},
cc = Count[
rows61C,
x_ /;
x["Target"] === "Continue" &&
MemberQ[x["States"], s]
];

ss = Count[
rows61C,
x_ /;
x["Target"] === "Stop" &&
MemberQ[x["States"], s]
];

<|
"State" → s,
"ContinueCases" → cc,
"StopCases" → ss,
"Conflict" → TrueQ[cc > 0 && ss > 0]
|>
],
{s, states61C}
];



Dataset[conf61C]
```

Out[844]=

State											ContinueCases	StopCases	Conf
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	2	0	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	0	2	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	2	0	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	0	2	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	2	0	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	0	2	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	2	0	False
{3, 1}	1 1 3 total	1 1	4	1	{1, 1} {1, 1} 3 total	1 total 1 total	1 total 1 total	{4, 1}	4 total	1 total	0	2	False

In[845]:=

```
cert61C = <|
"Stage" → "S61C",
"Radius" → 2,
"TrainCases" → Length[rows61C],
"StateTypes" → Length[conf61C],
"ConflictingStates" → Count[
conf61C,
x_ /; TrueQ[x["Conflict"]]
],
"PureContinueStates" → Count[
conf61C,
x_ /;
x["ContinueCases"] > 0 && x["StopCases"] == 0
],
"PureStopStates" → Count[
conf61C,
x_ /;
x["StopCases"] > 0 && x["ContinueCases"] == 0
],
"CoreTCCTChanged" → False,
"UnionSemantics" → "SingleFinalGlobalUnion"
|>;
```

```
Dataset[{cert61C}]
```

Out[846]=

Stage	Radius	TrainCases	StateTypes	ConflictingStates	PureContinueStates	PureStopStates	CoreTC
S61C	2	16	8	0	4	4	False

In[847]:=

```
ClearAll[DecisionStates62];


DecisionStates62[c_List] := Module[{rej, lev},
rej = Reject61[c];
If[Length[rej] == 0, Return[{}]];
lev = SigLevels61[c, 2][[3]];
DeleteDuplicates[Lookup[lev, rej[[All, 2]]]]
];


stateDict62 = {};


Do[
stateDict62 = Join[
stateDict62,
DecisionStates62[Case59[d, a, "Continue"]],
DecisionStates62[Case59[d, a, "Stop"]]
],
{d, {2, 5}},
{a, 1, 4}
];


stateDict62 = DeleteDuplicates[stateDict62];


Length[stateDict62]
```

Out[852]=

8

In[853]:=

```
ClearAll[Prep62];



Prep62[c_List] := Module[{p, lev, n, codes, pos, j},
p = Pack60[c];
n = p[[7]];
lev = SigLevels61[c, 2][[3]];

codes = Table[
pos = FirstPosition[
stateDict62,
Lookup[lev, j],
Missing["NF"]
];
If[MissingQ[pos], 0, First[pos]],
{j, n}
];

{p, codes}
];
```

In[855]:=

```
ClearAll[Run62];



Run62[x_List, pol_List, ret_Integer : 0] := Module[
{p = x[[1]], rc = x[[2]], pa, id, od, q, ca, ans, n, pin, pout, cin, cout,
h, z, h2, nz, arr, j, k, s, pp, ap, pz, o, zv, rf, rd, dd, pg, allow,
active, reached, pass, code, dc = {}},

pa = p[[1]];
id = p[[2]];
od = p[[3]];
q = p[[4]];
ca = p[[5]];
ans = p[[6]];
n = p[[7]];
pin = p[[8]];
pout = p[[9]];
cin = p[[10]];
cout = p[[11]];

h = ConstantArray[0, n];
```

```
z = ConstantArray[0, n];
arr = ConstantArray[-1, n];

h[[q]] = 1;
arr[[q]] = 0;

For[k = 1, k ≤ 64, k++,

h2 = ConstantArray[0, n];
nz = ConstantArray[0, n];
active = 0;

Do[
ap = 0;
pz = 0;

Do[
pp = pa[[j, s]];
If[
pp > 0 && h[[pp]] == 1,
ap++;
If[z[[pp]] > pz, pz = z[[pp]]]
],
{s, Length[pa[[j]]]}
];

If[
ap > 0,

o = Boole[ap > 1];

zv = {0, 1, 2, 1, 3, 1, 3, 1}[[1 + 2 * pz + o]];
nz[[j]] = zv;

If[arr[[j]] < 0, arr[[j]] = k];

rf = {
1,
Boole[zv > 0],
zv,
Sign[pin[[j]] - id[[j]]],
Sign[pout[[j]] - id[[j]]],
Sign[cin[[j]] - id[[j]]],
Sign[cout[[j]] - od[[j]]],
```

```
Sign[id[[j]] - od[[j]]],
Sign[pout[[j]] - cin[[j]]]
};

rd = rw60.rf;
dd = 1 - cin[[j]] + 2 * cout[[j]];
pg = od[[j]] + 2 * Boole[zv > 0] - zv;

code = rc[[j]];

If[
rd > 0 && dd ≤ 0,
AppendTo[dc, code]
];

allow = If[
rd > 0 && dd ≤ 0 && code > 0 && MemberQ[pol, code],
pg > 0,
If[rd > 0, dd > 0, pg > 0]
];

If[
TrueQ[allow],
h2[[j]] = 1;
active++
]
],
{j, n}
];

h = h2;
z = nz;

If[active == 0, Break[]]
];

reached = Select[
Range[4],
arr[[ca[[#]]]] ≥ 0 &
];

pass = Boole[
Length[reached] === 1 && First[reached] === ans
];
```

```
If[
ret == 1,
{pass, DeleteDuplicates[dc]},
pass
]
];
```

In[857]:=

```
trainSpec62 = Tuples[
{{2, 5}, Range[4], {"Continue", "Stop"}}
];
```

```
heldSpec62 = Tuples[
{{9, 15}, Range[4], {"Continue", "Stop"}}
];
```

```
train62 = Table[
Prep62[Case59 @@ trainSpec62[[i]]],
{i, Length[trainSpec62]}
];
```

```
held62 = Table[
Prep62[Case59 @@ heldSpec62[[i]]],
{i, Length[heldSpec62]}
];
```

```
{
Length[stateDict62],
Length[train62],
Length[held62],
2^Length[stateDict62]
}
```

Out[861]=

```
{8, 16, 16, 256}
```

In[862]:=

```
ClearAll[policies62, scores62, perfect62, bestScore62, best62];



policies62 = Subsets[Range[Length[stateDict62]]];



scores62 = Table[
Total[
Table[
Run62[train62[[i]], policies62[[j]]],
{i, 1, Length[train62]}
]
],
{j, 1, Length[policies62]}
];



{
Length[policies62],
Length[scores62],
VectorQ[scores62, IntegerQ],
Min[scores62],
Max[scores62]
}
```

Out[865]=

```
{256, 256, True, 0, 16}
```

In[866]:=

```
ClearAll[test62];



test62 = Table[
{
i,
Run62[train62[[i]], {}],
Run62[train62[[i]], Range[8]]
},
{i, 1, 16}
];



Dataset[
(<|
"Case" → #[[1]],
"Empty" → #[[2]],
"All" → #[[3]],
"EmptyHead" → Head[#[[2]]],
"AllHead" → Head[#[[3]]]
|> &) /@ test62
]
```

Out[868]=

Case	Empty	All	EmptyHead	AllHead
1	0	1	Integer	Integer
2	1	0	Integer	Integer
3	0	1	Integer	Integer
4	1	0	Integer	Integer
5	0	1	Integer	Integer
6	1	0	Integer	Integer
7	0	1	Integer	Integer
8	1	0	Integer	Integer
9	0	1	Integer	Integer
10	1	0	Integer	Integer
11	0	1	Integer	Integer
12	1	0	Integer	Integer
13	0	1	Integer	Integer
14	1	0	Integer	Integer
15	0	1	Integer	Integer
16	1	0	Integer	Integer

In[869]:=

```
{
Count[test62, {_, _Integer, _Integer}],
Select[
test62,
! IntegerQ[#[[2]]] || ! IntegerQ[#[[3]]] &
]
}
```

Out[869]=

```
{16, {}}
```

In[870]:=

```
{
ValueQ[train62],
Length[train62],
ValueQ[stateDict62],
Length[stateDict62],
ValueQ[rw60],
Length[DownValues[Run62]]
}
```

Out[870]=

```
{True, 16, True, 8, True, 1}
```

In[871]:=

```
ClearAll[Run62];
```

```
Run62[x_List, pol_List, ret_Integer : 0] := Module[
{p, rc, pa, id, od, q, ca, ans, n, pin, pout, cin, cout,
h, z, h2, nz, arr, j, k, s, pp, ap, pz, o, zv, rf, rd, dd, pg,
allow, active, reached, pass, code, dc = {}},

p = x[[1]];
rc = x[[2]];

pa = p[[1]];
id = p[[2]];
od = p[[3]];
q = p[[4]];
ca = p[[5]];
ans = p[[6]];
n = p[[7]];
pin = p[[8]];
pout = p[[9]];
cin = p[[10]];
cout = p[[11]];

h = ConstantArray[0, n];
z = ConstantArray[0, n];
arr = ConstantArray[-1, n];

h[[q]] = 1;
arr[[q]] = 0;

For[k = 1, k ≤ 64, k++,

h2 = ConstantArray[0, n];
nz = ConstantArray[0, n];
active = 0;

Do[
ap = 0;
pz = 0;

Do[
pp = pa[[j, s]];

If[
IntegerQ[pp] && pp > 0 && h[[pp]] == 1,
ap++;
```

```
If[z[[pp]] > pz, pz = z[[pp]]
],
{s, 1, Length[pa[[j]]]}
];

If[
ap > 0,

o = Boole[ap > 1];
zv = {0, 1, 2, 1, 3, 1, 3, 1}[[1 + 2 * pz + o]];
nz[[j]] = zv;

If[arr[[j]] < 0, arr[[j]] = k];

rf = {
1,
Boole[zv > 0],
zv,
Sign[pin[[j]] - id[[j]]],
Sign[pout[[j]] - id[[j]]],
Sign[cin[[j]] - id[[j]]],
Sign[cout[[j]] - od[[j]]],
Sign[id[[j]] - od[[j]]],
Sign[pout[[j]] - cin[[j]]]
};

rd = rw60.rf;
dd = 1 - cin[[j]] + 2 * cout[[j]];
pg = od[[j]] + 2 * Boole[zv > 0] - zv;
code = rc[[j]];

If[
rd > 0 && dd ≤ 0,
AppendTo[dc, code]
];

allow = If[
rd > 0 && dd ≤ 0 &&
IntegerQ[code] && code > 0 && MemberQ[pol, code],
pg > 0,
If[rd > 0, dd > 0, pg > 0]
];

If[
```

```
        TrueQ[allow],
        h2[[j]] = 1;
        active++
        ]
        ],
        {j, 1, n}
        ];

        h = h2;
        z = nz;

        If[active == 0, Break[]]
        ];

        reached = Select[
        Range[4],
        Function[u, arr[[ca[[u]]]] ≥ 0]
        ];

        pass = Boole[
        Length[reached] == 1 && First[reached] == ans
        ];

        If[
        ret == 1,
        {pass, DeleteDuplicates[dc]},
        pass
        ]
        ];
```

In[873]:=

```
        {
        Length[DownValues[Run62]],
        Run62[train62[[1]], {}],
        Run62[train62[[1]], Range[8]]
        }
```

Out[873]=

```
        {1, 0, 1}
```

In[874]:=

```
check62 = Flatten[
Table[
{
Run62[train62[[i]], {}],
Run62[train62[[i]], Range[8]]
},
{i, 1, 16}
]
];



{
Length[check62],
VectorQ[check62, IntegerQ],
DeleteDuplicates[check62]
}
```

Out[875]=

```
{32, True, {0, 1}}
```

In[879]:=
```
policies62 = Subsets[Range[8]];



scores62 = Table[
Total[
Table[
Run62[train62[[i]], policies62[[j]]],
{i, 1, 16}
]
],
{j, 1, 256}
];



{
Length[scores62],
VectorQ[scores62, IntegerQ],
Min[scores62],
Max[scores62]
}
```
Out[881]=
```
{256, True, 0, 16}
```

In[882]:=

```
perfect62 = Pick[
policies62,
scores62,
16
];



best62 = First@SortBy[
perfect62,
Length
];



<|
"PerfectPolicies" → Length[perfect62],
"SelectedStateCount" → Length[best62],
"SelectedCodes" → best62,
"SelectedStates" → stateDict62[[best62]]
|>
```

Out[884]=

```
<|PerfectPolicies → 1, SelectedStateCount → 4,
 SelectedCodes → {1, 3, 5, 7}, SelectedStates →
  {{{{3, 1}, {{1, 1}, {1, 1}, {1, 1}}, {{4, 1}}}, {{{1, 1}, {{1, 1}}, {{3, 1}}}, {{1, 1}, {{1, 1}}, {{3, 1}}},
     {{1, 1}, {{1, 1}}, {{3, 1}}}}, {{{4, 1}, {{0, 1}, {0, 4}, {1, 1}, {3, 1}}, {{1, 0}}}}},
   {{{3, 1}, {{1, 1}, {1, 1}, {1, 1}}, {{4, 1}}}, {{{1, 1}, {{1, 1}}, {{3, 1}}}, {{1, 1}, {{1, 1}}, {{3, 1}}},
     {{1, 1}, {{2, 1}}, {{3, 1}}}}, {{{4, 1}, {{0, 1}, {0, 4}, {1, 1}, {3, 1}}, {{1, 0}}}}},
   {{{3, 1}, {{1, 1}, {1, 1}, {1, 1}}, {{4, 1}}}, {{{1, 1}, {{1, 1}}, {{3, 1}}}, {{1, 1}, {{1, 1}}, {{3, 1}}},
     {{1, 1}, {{3, 1}}, {{3, 1}}}}, {{{4, 1}, {{0, 1}, {0, 4}, {1, 1}, {3, 1}}, {{1, 0}}}}},
   {{{3, 1}, {{1, 1}, {1, 1}, {1, 1}}, {{4, 1}}}, {{{1, 1}, {{1, 1}}, {{3, 1}}}, {{1, 1}, {{1, 1}}, {{3, 1}}},
     {{1, 1}, {{4, 1}}, {{3, 1}}}}, {{{4, 1}, {{0, 1}, {0, 4}, {1, 1}, {3, 1}}, {{1, 0}}}}}|>
```

In[885]:=

```
trainPassed62 = Total[
Run62[#, best62] & /@ train62
];



heldPassed62 = Total[
Run62[#, best62] & /@ held62
];



<|
"TrainPassed" → trainPassed62,
"TrainCases" → Length[train62],
"HeldoutPassed" → heldPassed62,
"HeldoutCases" → Length[held62],
"HeldoutAccuracy" → N[heldPassed62 / Length[held62]]
|>
```

Out[887]=

```
<|TrainPassed → 16, TrainCases → 16,
 HeldoutPassed → 16, HeldoutCases → 16, HeldoutAccuracy → 1.|>
```

In[888]:=

```
rows62 = MapThread[
Function[{sp, x},
<|
"Depth" → sp[[1]],
"Answer" → sp[[2]],
"Target" → sp[[3]],
"Pass" → Run62[x, best62]
|>
],
{heldSpec62, held62}
];



sum62 = Flatten[
Table[
Module[{z},
z = Select[
rows62,
#["Depth"] === d && #["Target"] === t &
];
<|
"Depth" → d,
"Target" → t,
"Cases" → Length[z],
"Passed" → Total[Lookup[z, "Pass"]]
|>
],
{d, {9, 15}},
{t, {"Continue", "Stop"}}
],
1
];



Dataset[sum62]
```

Out[890]=

Depth	Target	Cases	Passed
9	Continue	4	4
9	Stop	4	4
15	Continue	4	4
15	Stop	4	4

In[891]:=

```
heldCodes62 = DeleteDuplicates@Flatten[
Table[
Run62[held62[[i]], best62, 1][[2]],
{i, Length[held62]}
]
];
```

```
<|
"TrainStateTypes" → Length[stateDict62],
"SelectedCodes" → best62,
"HeldoutDecisionCodes" → heldCodes62,
"UnseenHeldoutState" → MemberQ[heldCodes62, 0]
|>
```

Out[892]=

```
<|TrainStateTypes → 8, SelectedCodes → {1, 3, 5, 7},
 HeldoutDecisionCodes → {1, 0, 2, 3, 4, 5, 6, 7, 8}, UnseenHeldoutState → True|>
```

In[893]:=

```
audit62 = Table[
Module[{r, codes},
r = Run62[held62[[i]], best62, 1];
codes = r[[2]];
<|
"Depth" → heldSpec62[[i, 1]],
"Answer" → heldSpec62[[i, 2]],
"Target" → heldSpec62[[i, 3]],
"Pass" → r[[1]],
"DecisionCodes" → codes,
"HasUnseen" → MemberQ[codes, 0],
"SelectedSeen" → Intersection[codes, best62]
|>
],
{i, Length[held62]}
];
```

```
Dataset[audit62]
```

Out[894]=

Depth	Answer	Target	Pass	DecisionCodes	HasUnseen	SelectedSeen
9	1	Continue	1	{1, 0}	True	{1}
9	1	Stop	1	{2}	False	{}
9	2	Continue	1	{3, 0}	True	{3}
9	2	Stop	1	{4}	False	{}
9	3	Continue	1	{5, 0}	True	{5}
9	3	Stop	1	{6}	False	{}
9	4	Continue	1	{7, 0}	True	{7}
9	4	Stop	1	{8}	False	{}
15	1	Continue	1	{1, 0}	True	{1}
15	1	Stop	1	{2}	False	{}
15	2	Continue	1	{3, 0}	True	{3}
15	2	Stop	1	{4}	False	{}
15	3	Continue	1	{5, 0}	True	{5}
15	3	Stop	1	{6}	False	{}
15	4	Continue	1	{7, 0}	True	{7}
15	4	Stop	1	{8}	False	{}

In[895]:=

```
sumSupport62 = Flatten[
Table[
Module[{z},
z = Select[
audit62,
#["Depth"] === d && #["Target"] === t &
];
<|
"Depth" → d,
"Target" → t,
"Cases" → Length[z],
"Passed" → Total[Lookup[z, "Pass"]],
"UnseenCases" → Count[
z, x_ /; TrueQ[x["HasUnseen"]]
],
"CasesUsingSelectedState" → Count[
z, x_ /; Length[x["SelectedSeen"]] > 0
]
|>
],
{d, {9, 15}},
{t, {"Continue", "Stop"}}
],
1
];
```

```
Dataset[sumSupport62]
```

Out[896]=

Depth	Target	Cases	Passed	UnseenCases	CasesUsingSelectedState
9	Continue	4	4	4	4
9	Stop	4	4	0	0
15	Continue	4	4	4	4
15	Stop	4	4	0	0

In[897]:=

```
cert62 = <|
"Stage" → "S62A",
"Benchmark" → "S59BalancedOppositeAction",
"Representation" → "AnonymousRadius2DecisionState",
"StateTypes" → 8,
"PoliciesTested" → 256,
"PerfectPolicies" → Length[perfect62],
"SelectedCodes" → best62,
"TrainCases" → 16,
"TrainPassed" → trainPassed62,
"TrainAccuracy" → N[trainPassed62 / 16],
"HeldoutCases" → 16,
"HeldoutPassed" → heldPassed62,
"HeldoutAccuracy" → N[heldPassed62/16],
"HeldoutDepths" → {9, 15},
"UnseenHeldoutDecisionState" → MemberQ[heldCodes62, 0],
"FinalOutcomeOnly" → True,
"IntermediateLabels" → 0,
"StopLabels" → 0,
"HeldoutRetraining" → False,
"PolicySearchOnHeldout" → False,
"CoreTCCTChanged" → False,
"UnionSemantics" → "SingleFinalGlobalUnion"
|>;


Dataset[{cert62}]
```

Out[898]=

Stage	S62A
Benchmark	S59BalancedOppositeAction
Representation	AnonymousRadius2DecisionState
StateTypes	8
PoliciesTested	256
PerfectPolicies	1
SelectedCodes	{1, 3, 5, 7}
TrainCases	16
TrainPassed	16
TrainAccuracy	1.0
HeldoutCases	16
HeldoutPassed	16
HeldoutAccuracy	1.0
HeldoutDepths	{9, 15}
UnseenHeldoutDecisionState	True
FinalOutcomeOnly	True
IntermediateLabels	0
StopLabels	0
HeldoutRetraining	False
PolicySearchOnHeldout	False

22 total